Online Computation Offloading for Collaborative Space/Aerial-Aided Edge Computing Toward 6G System

Yi Liu, Li Jiang, *Member, IEEE*, Qi Qi, *Senior Member, IEEE*, Kan Xie, *Member, IEEE*, and Shengli Xie, *Fellow, IEEE*

***Abstract*—In 6G systems, space-air-ground integrated network (SAGIN), relying on space/aerial communications to complement terrestrial networks, is developed to achieve worldwide connectivity and multi-service access especially in remote areas. However, the heavy workload of the resource-limited satellites may raise an important issue about reducing service coverage and quality. In this article, we propose a collaborative edge computing framework for SAGIN-aided 6G system, in which the LEO satellites are considered as both "servers" and "users". Excepting provide services for ground users/devices, the LEO satellites is able to offload the tasks to nearby aircrafts via one-hop link or offload them to the cloud server along multi-hop satellite path. To minimize the long-term task completion delay of LEO satellites, a stochastic optimization problem is formulated by considering the variation of the space/aerial environment. The Lyapunov-based optimization method is developed to solve the problem and the delayed online learning technique is adapted to predict dynamic task arrival and queue length of satellites and aircrafts. Numerical results confirm the effectiveness of the proposed collaborative offloading scheme for reducing tasks completion delay of LEO satellites while guaranteeing computation efficiency.**

***Index Terms*—Mobile edge computing, space-air-ground integrated network, collaborative offloading, online learning, 6G system.**

Manuscript received 17 May 2023; revised 11 July 2023; accepted 18 August 2023. Date of publication 7 September 2023; date of current version 13 February 2024. This work was supported in part by the National Key R&D Program of China under Grants 2020YFB1807805 and 2020YFB1807800 and in part by the Programs of NSFC under Grants U1911401, 61973087, and 62371142. The review of this article was coordinated by Dr. Zehui Xiong. *(Corresponding author : Li Jiang.)*

Yi Liu is with the School of Automation, Guangdong University of Technology, and Guangdong Province Key Lab. of IoT Information Technology, Guangzhou 510006, China (e-mail: yi.liu@gdut.edu.cn).

Li Jiang is with the School of Automation, Guangdong University of Technology, and Key Laboratory of Intelligent Detection and Internet of Manufacturing Things, Ministry of Education, Guangzhou 510006, China (e-mail: jiangli@gdut.edu.cn).

Qi Qi is with State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: qiqi8266@bupt.edu.cn).

Kan Xie is with the School of Automation, Guangdong University of Technology, and Guangdong-HongKong-Macao Joint Laboratory for Smart Discrete Manufacturing, Guangzhou 510006, China (e-mail: kxie@gdut.edu.cn).

Shengli Xie is with the 111 Center for Intelligent Batch Manufacturing Based on IoT Technology, and Key Laboratory of Intelligent Information Processing and System Integration of IoT, Ministry of Education of the P.R.C., Guangzhou 510006, China (e-mail: shlxie@gdut.edu.cn).

Digital Object Identifier 10.1109/TVT.2023.3312676

I. Introduction

With the gradual maturity and commercialization of the fifth generation (5G) wireless system, its shortcomings promote the research and development of the sixth generation (6G) wireless systems [1], [2], [3]. One of the examples is 5G system is still limited to cover some typical remote areas, such as desert, ocean, mountains, etc. which are difficult to be covered by ground base stations and promised to be obtained 100% coverage in 6G system. To achieve this goal, space-air-ground integrated networks (SAGIN) are raised to realize the ubiquitous access services for massive mobile users and Internet of Things (IoT) devices even in remote areas, maritime applications and emergency circumstances [5], [6]. In such network, space components includes geostationary earth orbit (GEO), medium earth orbit (MEO), and low earth orbit (LEO) satellites. Specially, the LEO satellites are widely used to establish near-earth satellite networks for massive access and data backhaul for remote area users with low cost of deployment and maintenance [7], [8]. The air components are unmanned aircrafts involving balloons, airships and airplanes positioned above 20 km altitude in the stratosphere, to provide lower delay and more stable connections for terrestrial users compared to satellite-terrestrial link. The ground components mainly consist of ground base stations and servers which are capable of providing high data-rate wireless service for users, but will fail in the case without terrestrial infrastructure [9].

Since 6G wireless communication system is expected to provide ubiquitous, high-quality, high-reliability and intelligence for full automation, the SAGIN are required not only to provide a complete global coverage but a new computing paradigm to support new intelligent applications and sophisticated services in different areas and scenarios [11]. Usually, cloud computing which enables elastic on-demand resource allocation, easy applications and diverse services provisioning, is the first candidate for SAGIN to handle the burdensome computation tasks. However, the long physical distance, limited communication bandwidth, intermittent network connectivity in SAGIN environment limits the operation of cloud computing in 6G system with many delay-sensitive applications. Owing to distributed resources allocation and management, mobile edge computing (MEC) is proposed to provide the cloud computing in close proximity to mobile ground users (at the edge of network) [12], [13],

[14]. The MEC has the advantage of handling delay-sensitive tasks at the local level, such as data collection, event monitoring, information extraction and so on [15], [16], [17]. Furthermore, cloud computing and MEC can complement each other to form an integrated multi-tier computing paradigm to support more intelligent applications and sophisticated services in SAGIN.

Recently, many researchers devoted to study the in-depth combination of the cloud/edge computing and SAGIN. Chen et al. [18] proposed a MEC-driven SAGIN in which the UAV is responsible for collecting tasks from IoT devices and forwarding to a BS or the cloud server through satellites network. Zhou et al. [19] considered the delay-oriented IoT services and investigated a computing task scheduling problem in SAGIN. Cheng et al. [20] presented a SAGIN edge/cloud computing architecture, and then studied a deep reinforcement learning based approach to learn the optimal offloading policy from the dynamic SAGIN environments. Yu et al. [21] designed an edge computing-enhanced SAGIN and proposed an offloading and caching algorithm to minimize the task completion time and satellite resource usage. Liu et al. [22] focused on the energy-efficient SAGIN edge computing in which the offloading decision is made based on UAV and satellites' energy level, communication conditions and computing capabilities. Liao et al. [23] jointly optimized the task offloading and resource allocation for Power IoT devices under several challenges in SAGIN such as incomplete information, dimensionality curse, etc.

In above studies, the LEO satellite is generally treated as an "edge server" in SAGIN edge computing to provide computing services to ground users/devices in remote areas. Actually, the resources (computation and energy) limited satellite should be more considered as a "user" which also needs computation services not only for ground requirements but its own exclusive applications. In this article, we focus on designing a collaborative space/aerial-aided edge computing framework, in which the LEO satellites in space domain are responsible for providing computation services to ground users/IoT devices in remote areas without the cellular network coverage. Meanwhile, the aircrafts (including UAVs, high-altitude platforms, and balloons) in aerial domain hovers at a certain area, are able to provide the computation cooperation to the nearby LEO satellite. Specifically, once the tasks collection and generation is finished, the LEO satellite can offload tasks to a nearby aircraft via one-hop communication or the cloud server via muli-hop satellites backhaul link, and make a decision about the ratio of local computing tasks and offloading tasks.

To design an efficient collaborative MEC paradigm for space/aerial-aided 6G system, some technical issues should be carefully addressed. The first issue comes from *variability of space/aerial network*. Since the satellites and aircrafts have different communication and computation capacities and their relative positions are varying over time, the transmission performance of satellites link and satellite-aircraft link may highly dynamic. Moreover, the tasks at satellites are composed by the tasks collected from ground and generated by their own applications, the task generation usually presents high variability mainly due to different workload levels or resource contention in SAGIN environments. The task queue length at each satellite and aircraft are also time-varying and difficult to determine at advance due to the excessive signaling overhead and privacy concerns.

The second issue is on *tradeoff between computation and communication resource consumptions* in the proposed framework. By exploiting cooperation between satellite and aircraft, and cooperation among satellites, tasks are allowed to be offloaded multiple hops away to cloud server or one hop to the nearby aircraft. It is obvious the former approach achieves lower computation latency but at the expense of higher transmission latency than that of the latter one. Hence, selecting the optimal offloading approach according to current conditions is critical to strike a good balance between computing and communication resource consumptions to obtain the optimal quality of services. This brings new modeling requirements for incorporating interplay and interdependency among the management of these resources.

By considering the above issues, we develop an online cooperative offloading and scheduling scheme in space/aerial-aided 6G system. We employ Lyapunov optimization to exploit spatial-temporal optimality of long-term completion delay minimization problem. The proposed scheme is devoted to an online offloading and scheduling policy by addressing the assignment problem for each satellite: where and how many tasks should be offloaded for the minimum completion delay in terms of computation capabilities? However, such offloading policy cannot become a reality without complete task arrival and queue length knowledge. To adapt to the variability of these two parameters, we resort to the delayed online learning method to predict task arrival and queue length of both satellites and aircrafts, which are used for the basis of offloading decision making. Then, the optimal offloading and scheduling decisions obtained via a bounded integer programming problem, are expected to minimize the long-term task completion delay.

The main contributions of this article are summarized as follows.

1) We propose a collaborative space/aerial-aided MEC framework for 6G system in which the satellites are able to offload the tasks to nearby aircrafts through one-hop connection or cloud server along multi-hop offloading path.
2) We formulate a stochastic optimization problem to minimize the long-term completion delay of the satellites. The Lyapunov-based optimization is developed to decompose the stochastic optimization problem into separate deterministic subproblem for each satellite.
3) With the aim of minimizing the loss due to prediction errors over time, we adapt the delayed online learning technique to facilitate task arrival and queue length prediction, which is fed as input to Lyapunov-based cooperative offloading policy. We achieve queue awareness by continuously adjusting task offloading and scheduling strategies in accordance with predicted queue information, which balances the tradeoff between completion delay minimization and queuing delay reduction.

The remainder of this article is organized as follows. Section II introduces the system model of the collaborative

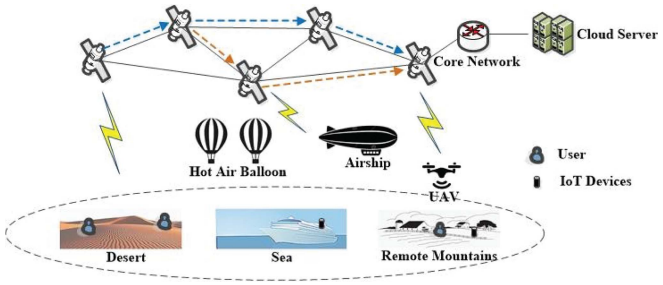

Fig. 1. System model of SAGECN.

space/aerial MEC network. The cooperative computation offloading and scheduling problem is formulated in Section III. The online collaborative offloading mechanism is presented in Section IV. Section V shows the numerical results of the proposed methods. Section VI concludes this article.

II. System Model

A. Space/Aeria-Aided MEC Model

We consider a space/aerial-aided MEC network, as shown in Fig. 1, consisting of a set of LEO satellites $\mathbb{N} = \{1, \ldots, N\}$ and a set of aircrafts $\mathbb{U} = \{1, \ldots, U\}$ which can provide computation services for the ground users without cellular coverage. The LEO satellites and aircrafts which can be hot air balloons, airship, fixed-wing UAVs, rotary-wing UAVs, etc, are endowed with computing capacity and capable of serving the user-generated computation tasks on the ground. The LEO satellites in the proposed network are connected by backhaul links, which can be used to send task requests or replies among satellites. By exploiting cooperation among LEO satellites and aircrafts, the arrival tasks at one LEO satellite can be processed locally, offloaded to the nearest aircrafts, or offloaded multiple hops to the cloud server via satellite backhaul links. The process of the cooperative operations are time-slotted, where $t \in \{1, \ldots, T\}$.

For each satellite $n \in \mathbb{N}$, let $\alpha_n(t) = \{0, 1\}$ denote the offloading decision of satellite n at time slot t, $\alpha_n(t) = 0$ represents the local computing, $\alpha_n(t) = 1$ represents the satellite n decides to offload task. Then, we use binary variables $\beta_{n,m}(t) = \{0, 1\}$ and $\theta_{n,u}(t) = \{0, 1\}$ to denote the cooperative offloading decision for satellite n at each time slot t. Specifically, $\beta_{n,m}(t) = 1$ represents that the task of satellite n is offloaded to the neighbor satellite m which will be added in inter-satellite offloading path, and $\beta_{n,m}(t) = 0$ otherwise. $\theta_{n,u}(t) = 1$ means satellite n offloads the task to the aircraft u for processing, and $\theta_{n,u} = 0$ otherwise.

B. Online Offloading Model

The nth LEO satellite generates the computing tasks (by itself or collected from ground users) at the beginning of each time slot, which can be specified by the tuple $(\rho^n, \sigma^n, \tau^n)$. ρ^n is the task size of a task, σ^n is the number of CPU cycles required for calculating one-bit task, τ^n is the deadline of the computing task at the nth satellite. At the beginning of each time slot, the satellite collaboratively reports the computing deadline τ^n and capture the maximum completion latency. For each computing task, the satellite can directly compute it, or forward it to other aircrafts or cloud sever for computation.

1) Satellite to Aircrafts: The LEO satellite can offload tasks to the nearest aircrafts $u \in \mathbb{N}$ through satellite-aircraft link. Without loss of generality, the channel gain of the satellite-aircraft link is set as a constant value g. Let $r_{n,u}$ denote the data rate of satellite-aircraft link between the nth satellite and the uth aircraft, we have

$$r_{n,u} = B_{n,u} \log_2 \left(1 + \frac{p_n \mid g \mid^2}{\xi_S^2}\right) \tag{1}$$

where $B_{n,u}$ is the bandwidth of the satellite-aircraft link. p_n is the transmission power of the nth LEO satellite. ξ_S^2 is the power of noise in the satellite-aircraft communication scenarios, respectively.

Let d_S denote the propagation delay of the satellite-aircraft link. Considering $k_{n,u}(t)$ number of tasks is offloaded to the aircraft u by the nth LEO satellite at time slot t. The transmission delay of offloading tasks from satellite n to aircraft u can be obtained as

$$d_{n,u}^{SA}(t) = \alpha_n(t)\theta_{n,u}(t)\left(\frac{k_{n,u}(t)\rho^n}{r_{n,u}} + d_S\right). \tag{2}$$

2) Multi-Hop Offloading Via Satellite Propagation: Each LEO satellite can forward the computing task to the cloud server via multi-hop offloading path among other satellites in $\mathbb{N}$ (i.e., cooperative offloading). For each task, we use $L \in \{0, 1, \cdots\}$ denote the number of hops that the LEO satellite offload the task to the cloud server. In each hop, the peak gain of transceivers of satellite n in the direction of satellite m is given by G. Then, the data rate $r_{n,m}$ of the satellite link can be calculated as

$$r_{n,m} = B_{n,m} \log_2 \left(1 + \frac{p_n G^2}{\kappa_B B_{n,m} W(nm)}\right), \tag{3}$$

where $B_{n,m}$ is the bandwidth between satellite n and m, p_n is the transmission power of satellite n, κ_B is the Boltzmann constant, $W(nm)$ is the free-space path loss which is derived in [24]. Hence, the transmission time of offloading task from satellite n to satellite m can be written as

$$d_{n,m}(t) = \alpha_n(t)\beta_{n,m}(t)\frac{k_{n,m}(t)\rho^n}{r_{n,m}}, \tag{4}$$

where $k_{n,m}(t)$ denote the number of tasks offloaded from satellite n to satellite m.

Let $\mathbb{N}_n$ denote the set of LEO satellites contributing to the offloading service for satellite n, and $\Upsilon = \{n_1, \cdots, n_L\}$ denote the permutation of all L satellites along the offloading path. The task request from satellite n is offloaded L hops to cloud sever n_L for processing. Accordingly, the transmission delay of such offloading can be expressed as

$$d_n^{ML}(t) = \sum_{l \in L} d_{n_l, n_{l-1}}(t) \tag{5}$$

C. Task Computing Model

1) Satellite Computing Model: Let $\chi_n(t)$ denote the service limitation of computing components in satellite n which is

determined by the available resources at time slot t. The satellite is considered as resource limited. Since the computing capacity of satellite n is depending on the workloads and computing resources, we can obtain the task processing rate (in CUP cycles per second) of its computing components as

$$c_n(t) = \frac{1}{y} x^{\chi_n(t) - Q_n(t)}. \tag{6}$$

where $Q_n(t)$ is the amount of tasks buffered in satellite n at the beginning of slot t. x stands for the relationship between processing rate and workload/resource levels, y is the speed when the computing component of satellite is fully loaded. Due to the different structure of the computing components, parameters x and y of satellites are different. According to (6), satellite n can achieve larger processing rate as less workloads and more available resources.

Let $s_n(t)$ denote the amount of arrival tasks of satellite n at time slot t. Considering the number of the tasks processed by the satellite n is denoted as $K_n(t)$ at time slot t, we have

$$K_n(t) = \min\left\{\frac{c_n(t)}{\sigma_n Q_n(t)}, Q_n(t)\right\}. \tag{7}$$

The computing latency d_n of a task computed by satellite n includes the local computing delay and cloud sever computing delay, which can be expressed as

$$d_n^C(t) = (1 - \alpha_n(t))\frac{k_n(t)\sigma_n}{c_n(t)} + \sum_{l \in L} \alpha_{n_l}(t)\beta_{n_l, n_{l-1}}(t)\frac{Q_n(t)\sigma_n}{c_{cloud}}, \tag{8}$$

where c_{cloud} is the computing rate of the cloud server.

2) Aircraft Computing Model: Let $Q_u(t)$ denote the number of tasks queued in aircraft u at the beginning of time slot t. We can obtain the task processing rate of the aircraft u as

$$c_u(t) = \frac{1}{y} x^{\chi_u(t) - Q_u(t)}, \tag{9}$$

where $\chi_u(t)$ is the service limitation of computing components in aircraft u at time slot t. Let $k_{n,u}(t)$ denote the number of the tasks offloaded from satellite n to the aircraft u during time slot t. Since each aircraft needs to finish the tasks as soon as possible while guaranteeing the fairness, all tasks buffered in the same aircraft share the same resources without preemption. Then, the number of the tasks processed by the aircraft u for the satellites can be obtained as

$$K_u(t) = \min\left\{\frac{c_u(t)}{\sigma_n Q_u(t)}, Q_u(t)\right\}. \tag{10}$$

If the satellite n decides to offload tasks $k_{n,u}(t)$ to aircraft u for processing, the computing latency $d_{n,u}(t)$ can be obtained as

$$d_{n,u}^C(t) = \alpha_n(t)\theta_{n,u}(t)\frac{k_{n,u}(t)}{c_u(t)}. \tag{11}$$

D. Task Queue Model

We consider that the satellites and aircrafts offload and execute the tasks at the beginning of the time slot t, while the tasks collection which includes the new tasks arrival or offloading from other satellites happens at the end. We can obtain the number of tasks processed by satellite n at time slot t as

$$I_n(t) = \alpha_n(t)K_n(t) + \beta_{n,m}(t)k_{n,m}(t) + \theta_{n,u}(t)k_{n,u}(t), \tag{12}$$

where $\alpha_n(t) + \beta_{n,m} + \theta_{n,u} \leq 1$. Accordingly, the dynamics of task queues $Q_n(t)$ and $Q_u(t)$ associated with any satellite $n \in \mathbb{N}$ and aircraft $u \in \mathbb{U}$, respectively, can be described as

$$Q_n(t+1) = \max\{Q_n(t) - I_n(t), 0\} + s_n(t) + \sum_{m \in \mathbb{N}} k_{m,n}(t), \tag{13}$$

$$Q_u(t+1) = \max\{Q_u(t) - K_u(t), 0\} + \tilde{s}_u(t) + \sum_{n \in \mathbb{N}} k_{n,u}(t), \tag{14}$$

where $\sum_{m \in \mathbb{N}} k_{m,n}(t)$ is the number of tasks offloaded to n from satellite $m \in \mathbb{N}$. The first term on the right-hand-side of (13) and (14) captures the unprocessed tasks at time slot t after part of tasks are locally processed or offloaded away, and the last two terms describe the tasks arrived locally and offloaded from neighbor satellites.

Stability Constraint: The proposed network is stable only if it has a bounded time-averaged backlog [25], i.e.,

$$\bar{Q}_{sys} = \limsup_{T \to \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\{Q_n(t) + Q_u(t)\} < \infty, \forall n \in \mathbb{N}, \forall u \in \mathbb{U}. \tag{15}$$

III. Problem Formulation

The desired offloading mechanism is able to serve tasks with collaborative computing among satellites and aircrafts, and satellites' scheduling capacities. We focus on providing stable quality of service (QoS) of space/aerial-aided MEC network in terms of completion delay which includes transmission delay and computation delay.

Under proposed offloading mechanism, the transmission delay of satellite n at time slot t, denoted as $D_n^T(t)$, for transmitting tasks through satellite-aircraft links or multi-hop satellite link can be obtained as

$$D_n^T(t) = \sum_{u \in \mathbb{U}} d_{n,u}^{SA}(t) + d_n^{ML}(t). \tag{16}$$

Let $D_n^C(t)$ denote the computation delay of satellite n at time slot t, we have

$$D_n^C(t) = d_n^C(t) + \sum_{u \in \mathbb{U}} d_{n,u}^C(t). \tag{17}$$

Therefore, the instantaneous completion delay can be expressed as

$$D(t) = \sum_{n \in \mathbb{N}} (D_n^T(t) + D_n^C(t)). \tag{18}$$

Since our goal is to minimize long-term completion delay, the mathematical formulation of the collaborative offloading

problem in space/aerial-aided MEC network is given by

$$(\mathbf{P1}) \quad \min_{\substack{\alpha_n(t),\beta_{n,m}(t),\theta_{n,u}(t),\\ k_{n,m}(t),k_{n,u}(t),\forall n,u,t}} \lim_{T\to\infty} \frac{1}{T}\left[D(t)\right]$$

$$\text{s.t.} \quad C1 \quad \alpha_n(t) + \sum_{m\in\mathbb{N}} \beta_{n,m}(t) + \sum_{u\in\mathbb{U}} \theta_{n,u}(t) \le 1, n \in \mathbb{N},$$

$$C2 \quad \sum_{m\in\mathbb{N}} k_{m,n}(t) \le J^S_{\max}, \forall t$$

$$C3 \quad \sum_{u\in\mathbb{U}} k_{n,u}(t) \le J^U_{\max}, \forall t$$

where C1 is the constraint for satellite n can either process the task locally or offload it to the neighboring aircrafts or satellites at time slot t, C2 and C3 constraint the most number of tasks dispatched to the satellites and aircrafts for guaranteed the processing rate, respectively.

To solve problem (**P1**), satellites should make the offloading and tasks dispatch decisions during a time slot which are spatial-temporal coupled decisions. Moreover, the stochastic task arrivals at satellites and aircrafts make (**P1**) to be stochastic problem which is difficult to collect complete offline information of the workload levels at both satellite and aircraft. Hence, we develop an online optimization method to perform collaborative offloading based on task arrivals and workload prediction knowledge.

IV. Online Collaborative Offloading Mechanism

In this section, we firstly apply the Lyapunov optimization to decouple (**P1**) into per-frame deterministic problems. Then, taking task arrivals and workload variability into account, an asynchronous prediction policy is implemented based on delayed online learning.

A. Lyapunov Optimization

Let $\Theta(t) = \{\mathbf{Q}^N(t), \mathbf{Q}^U(t)\}$ denote the aggregate queue vector, where $\mathbf{Q}^N(t) = \{Q_n(t)\}_{n=1}^N$ and $\mathbf{Q}^U(t) = \{Q_u(t)\}_{u=1}^U$. To ensure both delay bounds of the satellite and aircraft, we define the following Lyapunov function to measure the congestion in the queues,

$$L(\Theta(t)) = \frac{1}{2}\left[\sum_{n=1}^{N} Q_n^2(t) + \sum_{n=1}^{U} Q_u^2(t)\right].$$

The conditional 1-slot Lyapunov drift is defined as,

$$\Delta(\Theta(t)) = E\Big\{L(\Theta(t+1)) - L(\Theta(t))|\Theta(t)\Big\}.$$

We incorporate the system delay into Lyapunov drift to guarantee the network stability and delay jointly. We have the following drift plus penalty function (using the drift plus penalty function framework developed in [25]),

$$\Delta(\Theta(t)) + VE\Big\{\sum_{n=1}^{N} D_n(t)|\Theta(t)\Big\}$$

where $V > 0$ is a parameter to effect the performance delay tradeoff. Again, using the techniques developed in [25], we can show that this function is bounded as,

$$\begin{aligned}
&\Delta(\Theta(t)) + V\mathbb{E}\Big\{D(t)|\Theta(t)\Big\} \le B_1 \\
&+ B_2 + V\mathbb{E}\Big\{\sum_{n=1}^{N} D_n(t)|\Theta(t)\Big\} \\
&+ \sum_{n=1}^{N} Q_n(t)\mathbb{E}\left\{\left(s_n(t) + \sum_{m\in\mathbb{N}} k_{m,n}(t) - I_n(t)\right)|\Theta(t)\right\} \\
&+ \sum_{u=1}^{U} Q_u(t)\mathbb{E}\left\{\left(\tilde{s}_u(t) + \sum_{n\in\mathbb{N}} k_{n,u}(t) - K_u(t)\right)|\Theta(t)\right\},
\end{aligned} \tag{19}$$

where B_1 and B_2 are constant and can be obtained as follows.

$$\begin{aligned}
&\frac{1}{2}\sum_{n=1}^{N}\mathbb{E}\left\{\left(s_n(t) + \sum_{m\in\mathbb{N}} k_{m,n}(t) - I_n(t)\right)^2\right\} \\
&\le \frac{1}{2}\sum_{n=1}^{N}\mathbb{E}\Big\{s_n^2(t) + \sum_{m\in\mathbb{N}} k_{m,n}^2(t) + I_n^2(t)\Big\} \le \frac{1}{2}\sum_{n=1}^{N}\Bigg[S_{\max}^2 \\
&+ (N+1)J_{\max}^2 + J_{\max,U}^2 + \left(\frac{c_n^{\max}}{\sigma_n^{\max}Q_n^{\max}}\right)^2\Bigg] \triangleq B_1
\end{aligned}$$

where $S_{\max}$ is the maximum number of the arrival tasks at each time slot, $\sum_{m\in\mathbb{N}} k_{m,n} \le J_{\max}$ specifies the limited offloading capacity of satellite n for satellite m by placing an upper bound $J_{\max}$, $J^U_{\max}$ is the upper bound of the offloading capacity that aircraft u for satellites. Similarly, we have the following definition of B_2

$$\begin{aligned}
&\frac{1}{2}\sum_{n=1}^{N}\mathbb{E}\left\{\left(\tilde{s}_u(t) + \sum_{n\in\mathbb{N}} k_{n,u}(t) - K_u(t)\right)^2\right\} \\
&\le \frac{1}{2}\sum_{n=1}^{N}\mathbb{E}\left\{(S^U_{\max})^2 + NJ^2_{\max} + \frac{c_u^{\max}}{\sigma_u^{\max}Q_u^{\max}}\right\} \triangleq B_2.
\end{aligned}$$

Let $\boldsymbol{a}_n(t) = \{\alpha_n(t), \beta_{n,m}(t), \theta_{n,u}(t)\}$ and $\boldsymbol{k}_n(t) = \{k_{n,m}(t), k_{n,u}(t)\}$. Now instead of solving the original problem, we minimize this bound on the drift plus penalty function.

$$\begin{aligned}
(\mathbf{P2}) \quad &\min_{\boldsymbol{a}_n(t),\boldsymbol{k}_n(t)} \quad V\mathbb{E}\left\{\sum_{n=1}^{N} D_n(t)|\Theta(t)\right\} \\
&+ \mathbb{E}\left\{\sum_{n=1}^{N} Q_n(t)\left(s_n(t) + \sum_{m\in\mathbb{N}} k_{m,n}(t) - I_n(t)\right)\right. \\
&\left.+ \sum_{u=1}^{U} Q_u(t)\left(\tilde{s}_u(t) + \sum_{n\in\mathbb{N}} k_{n,u}(t) - K_u(t)\right)|\Theta(t)\right\},
\end{aligned}$$

The problem (**P2**) can not be optimized without knowing the global knowledge of all satellites and aircrafts, e.g., all queued tasks in associated aircrafts $Q_u(t), \forall\mathbb{U}$, the arriving tasks $s_n(t)$

and $s_u(t)$. However, as the tasks transmission and processing at the space/aerial environment, the knowledge presents high variability due to varying workload and resource levels, especially under stochastic task arrivals. Hence, we adopt an online learning-aided cooperative offloading approach that employs online learning techniques to acquire prediction knowledge of arriving tasks and workload levels, which denoted as $P(t) = \{s_n(t), s_u(t), Q_n(t), Q_u(t) \mid n \in \mathbb{N}, u \in \mathbb{U}\}$.

B. Asynchronous Online Learning Method

1) Online Learning Method: In the online learning method, we consider a learning agent is capable of interacting the proposed network environment and collect concerned task arriving and workload information $P(t)$ for a period of time. Generally, the learner predicts $\hat{P}(t)$ and incurs a loss function $f_t(P(t))$ at each time slot t. The loss function can be derived as

$$f_t(\hat{P}(t)) = |\hat{P}(t) - P(t)|,$$

where $P(t)$ is the concerned information can be collected at the beginning of time slot t. To minimize the prediction error, we can construct a loss minimization problem as follows

$$\min \sum_{t \in T} f_t(\hat{P}(t))$$

The loss minimization problem can be solved by employing online gradient descent (OGD) method, in which the gradient of the objective function is used to approximate the best predictor. Specifically, the learning agent firstly computes the gradient of the loss function at $P(t)$, which denoted by $\nabla f_t|_{P(t)}$. Then, we can derive $P(t)$ in the subsequent time slot as

$$P(t+1) = P(t) - \eta \nabla f_t|_{P(t)}.$$

Based on OGD method, the loss function $f_t(P(t))$ is delivered and applied before operating next prediction in time slot $t+1$. However, in the proposed prediction strategy, the prediction of $P(t)$ at time slot t is highly relying on the complete information collection of all satellites and aircrafts which may only be achieved in future consecutive time slots. That is, the asynchronous processing of the satellites and aircrafts will make delayed feedback of the information which cannot be collected after a delay of several slots.

2) Asynchronous Online Learning Based Prediction: To handle the aforementioned delay issue, we employ delayed online gradient descent (DOGD) method, where loss function can be applied in delayed time slots instead of being delivered before next prediction. Formally, each slot t has a non-negative integer delay d_t. The feedback from t is delivered at the end of $t + d_t - 1$ and can be used in $t + d_t$. We consider a prediction window $\boldsymbol{T}_t = \{t+1, \ldots, t+t^{\max}\}$, where $t^{\max}$ is the deadline for collection all workload information. The actual workload $P(\tau)$ can be observed at the beginning of slot τ, and the learning agent predicts works information, $\hat{P}(t) = \{\hat{P}(\tau), \tau \in \boldsymbol{T}_t\}$ at slot t. During the prediction window $\boldsymbol{T}_t$, the loss function generated at time slot t is delivered at the end of $\tau - 1$ and can be used at τ. For each slot τ, the loss function can be obtained as

$$f_\tau(\hat{P}(\tau)) = |\hat{P}(\tau) - P(\tau)| \tag{20}$$

where $f_\tau(\cdot)$ is a convex function and $\hat{P}(\tau) \in [0, P^{\max}]$, $P^{\max} = \max_{t \in T} P(t)$. Then, the loss minimization problem over all slots can be derived as

$$\min_{\hat{P}(\tau) \in [0, P^{\max}]} \sum_{t \in T} \sum_{\tau \in \boldsymbol{T}_t} f_\tau(\hat{P}(\tau)) \tag{21}$$

According to DOGD method, the learning agent makes a workload prediction $\hat{P}(t)$ at any time slot t for each future slot $\tau \in \boldsymbol{T}_t$ based on the feedback that it has observed from t, and suffers the loss $\hat{P}(\tau)$. Next, we describe the update rule for each slot $\tau \in \boldsymbol{T}_t$ as:

$$\hat{P}(\tau+1) = \hat{P}(\tau) - \eta' \nabla f_t|_{P(t)} \tag{22}$$

where step size η' is typically set proportional to $\frac{1}{\sqrt{T+\Phi}}$ with $\Phi = \sum_{t \in T} \sum_{\tau \in \boldsymbol{T}_t} d_\tau$ denote the sum of all delays.

3) Regret Analysis: We next analyze the performance of the prediction of $\hat{P}(t)$ by computing an expected regret over random choices of $\hat{P}(t)$'s. The expected regret is computed by comparing the overall loss (objective value of (21)) incurred by our algorithm and by the best static prediction strategy [26]. Let $P^*(t)$ be the best static predictor. We can compute the expected regret as

$$\mathbf{REG}_T = \sum_{t \in T} \sum_{\tau \in \mathbf{T}_t} [f_\tau(\hat{P}(\tau)) - f_\tau(P^*(\tau))] \tag{23}$$

To give the upper-bounds of overall regret, we have the following theorem according to [27].

Theorem 1: Compared to the best static prediction strategy that uses $P^*(t)$, the regret of the proposed Algorithm 1, as for all $\forall t \in \mathbf{T}$, is upper-bounded by:

$$\mathbf{REG}_T \leq \frac{t^{\max}}{2\eta'} + \eta' \left[\frac{T t^{\max}}{2} + 2\Omega \right] \tag{24}$$

Algorithm 1: Online Offloading and Scheduling Algorithm.

1: **for** *each satellite* $n \in \mathbb{N}$ **do** ;
2: $P_n^{\max} \leftarrow \max\{s_n(t), s_u(t), Q_n(t), Q_u(t) \mid n \in \mathbb{N}, u \in \mathbb{U}\}$;
3: Derive

$$\eta' = \frac{P_n^{\max}}{\sqrt{T+\Phi}};$$

4: **for** *each* $\tau \in \mathbf{T}_t$ **do**;
5: Derive predicted knowledge $\hat{P}_n(t)$ according to:

$$\hat{P}(\tau+1) = \hat{P}(\tau) - \eta' \nabla f_t|_{P(t)}$$

5: **for** *each satellite* $m \in \mathbb{N}_n \vee \{n\}$ **do**;

$$c_n(t) = \frac{1}{y} x^{\chi_n(t) - \hat{Q}_n(t)}$$

6: **for** *each aircraft* $u \in \mathbb{U}$ **do** ;

$$c_u(t) = \frac{1}{y} x^{\chi_u(t) - \hat{Q}_u(t)}$$

V. Online Task Scheduling Policy

Given task arrivals and workload prediction knowledge, we employ online task scheduling policy to solve per-slot task scheduling problem for individual satellite.

A. Task Offloading and Scheduling Policy

In the proposed method, each satellite is responsible for offloading and scheduling tasks to aircrafts or cloud server with multi-hop inter-satellite communications. We note that the scheduling decisions (including local computing and cooperative offloading) of different satellites are independent from each other. For satellite n, the task offloading decisions $\boldsymbol{a}_n(t)$ and scheduling decisions $\boldsymbol{k}_n(t)$ can be obtained by solving

$$\begin{aligned}(\mathbf{P3}) \quad & \min_{\boldsymbol{a}_n(t),\boldsymbol{k}_n(t)} \quad V\mathbb{E}\left\{D_n(t)|\Theta(t)\right\} \\ & + \mathbb{E}\left\{\hat{Q}_n(t)\left(\hat{s}_n(t)+\sum_{m\in\mathbb{N}}k_{m,n}(t)-I_n(t)\right)\right. \\ & \left. + \sum_{u=1}^{U}\hat{Q}_u(t)\left(\hat{s}_u(t)+\sum_{n\in\mathbb{N}}k_{n,u}(t)-K_u(t)\right)|\Theta(t)\right\}, \\ \text{s.t.} \quad & C1, C2, C3. \end{aligned} \tag{25}$$

For satellite n, the task offloading and scheduling problem (**P3**) is a bounded integer programming problem, where constraints C1, C2, and C3 guarantee the tasks are properly scheduled. We develop branch-and-bound (BnB) method to solve this problem. The basic concept of such method is to set up a search tree in which the root node is problem (**P3**), and constantly search and update the upper and lower bound of the problem. The algorithm terminates as the upper and lower value is equal. Otherwise, BnB reselects the node to partition and repeats the search process.

Then, a BnB-based algorithm is designed to solve problem (**P3**): Firstly, the binary variables $\boldsymbol{a}_n(t)=\{\alpha_n(t), \beta_{n,m}(t), \theta_{n,u}(t)\}$ and integer variables $\boldsymbol{k}_n(t)=\{k_{n,m}(t), k_{n,u}(t)\}$ of (**P3**) are relaxed into continuous variables at the root problem (**RP**). In the (**RP**), the relaxed binary and integer variables are bounded by $0\le\alpha_n(t),\beta_{n,m}(t),\theta_{n,u}(t)\le 1$, $0\le k_{n,m}(t),k_{n,u}(t)\le J$. Then, the (**RP**) can be expressed as follows:

$$\begin{aligned}(\mathbf{RP}) \quad & \min_{\boldsymbol{a}_n(t),\boldsymbol{k}_n(t)} \quad F(\boldsymbol{a}_n(t),\boldsymbol{k}_n(t)) \\ & s.t. \quad C1, C2, C3, and \\ & \quad 0\le\alpha_n(t),\beta_{n,m}(t),\theta_{n,u}(t)\le 1, \\ & \quad 0\le k_{n,m}(t),k_{n,u}(t)\le J. \end{aligned} \tag{26}$$

where $F(\boldsymbol{a}_n(t),\boldsymbol{k}_n(t))=V\mathbb{E}\{D_n(t)|\Theta(t)\}+\mathbb{E}\Big\{\hat{Q}_n(t)(\hat{s}_n(t)+\sum_{m\in\mathbb{N}}k_{m,n}(t)-I_n(t))+\sum_{u=1}^{U}\hat{Q}_u(t)(\hat{s}_u(t)+\sum_{n\in\mathbb{N}}k_{n,u}(t)-K_u(t))|\Theta(t)\Big\}$.

Then, at each iteration, the relaxed problem (**RP**), which is tractable, can be solved to obtain the optimal value F^* and the optimal solution $\boldsymbol{a}^*$ and $\boldsymbol{k}^*$. Before feasible solution for all variables are obtained, BnB-based algorithm will divide (**RP**) into subproblems, i.e., branch problems. In this case, the left branch problem at pruning node ω (where $\omega=0$ for the **RP** problem) is formed as

$$\mathbf{RP}_\omega^l : \left\{\min_{\boldsymbol{a},\boldsymbol{k}} F(\boldsymbol{a},\boldsymbol{k}) : \boldsymbol{a},\boldsymbol{k}\in C_\omega^l\right\} \tag{27}$$

where $C_\omega^l = C_\omega \bigcap\{\boldsymbol{a},\boldsymbol{k} : x\le x^*, x\in\{\boldsymbol{a},\boldsymbol{k}\}, x^*\in\{\boldsymbol{a}^*,\boldsymbol{k}^*\}\}$, $C_\omega=\{C1, C2, C3, 0\le\alpha_n(t),\beta_{n,m}(t),\theta_{n,u}(t)\le 1, 0\le k_{n,m}(t),k_{n,u}(t)\le J\}$. Similarly, the right branch problem at pruning node ω is formed as

$$\mathbf{RP}_\omega^r : \left\{\min_{\boldsymbol{a},\boldsymbol{k}} F(\boldsymbol{a},\boldsymbol{k}) : \boldsymbol{a},\boldsymbol{k}\in C_\omega^r\right\} \tag{28}$$

where $C_\omega^r = C_\omega\bigcap\{\boldsymbol{a},\boldsymbol{k} : x\ge x^*+1\}$.

These steps are repeated until every element of $\{\boldsymbol{a},\boldsymbol{k}\}$ is integer. A node is pruned if its optimal value F^* is less than the current best feasible value of $F(\boldsymbol{a},\boldsymbol{k})$, or $\mathbf{RP}_\omega$ has no feasible solutions, or the all elements in $\{\boldsymbol{a},\boldsymbol{k}\}$ is integer. Finally, iteration will be terminated when there are no remaining subproblems.

Algorithm 2: Online Offloading and Scheduling Algorithm.

Knowledge Prediction Process
for *each satellite* $n\in\mathbb{N}$ **do** ;
Derive processing rate $c_n(t)$ according to (6):
for *each aircraft* $u\in\mathbb{U}$ **do** ;
Derive processing rate $c_u(t)$ according to (9):
for *each satellite* $m\in\mathbb{N}_n\vee\{n\}$ **do**;
Obtain predicted knowledge $\hat{P}(t)=\{\hat{s}_n(t),\hat{s}_u(t),\hat{Q}_n(t),\hat{Q}_u(t)\mid n\in\mathbb{N},u\in\mathbb{U}\}$ by applying Algorithm 1.;
Set $F^*=\infty$, Ω: the set of leaf nodes
Solve root problem (**RP**) at $\omega=0$
if every elements in $\{\boldsymbol{a},\boldsymbol{k}\}$ is integer **then**
Store $\boldsymbol{a}^*,\boldsymbol{k}^*$ and F^*, **return**
end if
while $\Omega\neq\varnothing$ **do**
  for RP $\in\{\mathbf{RP}_\omega^l,\mathbf{RP}_\omega^r:\omega\in\Omega\}$ **do**
  solve subproblem **RP**
    if RP is feasible **then**
    Obtain optimal solution $\boldsymbol{a}',\boldsymbol{k}'$ and objective value F'
      if $F'<F^*$ **then**
      store $\boldsymbol{a}',\boldsymbol{k}'$ and set $F^*=F'$
      else if $F'>F^*$ **then**
      Prune the branch
      end if
    else prune the node
    end if
  end for
end while

B. Online Offloading and Scheduling Algorithm

The proposed online offloading and scheduling (OOS) algorithm, which is summarized in Algorithm 2, consists of two

TABLE I
SYSTEM PARAMETERS

Parameter	Value	Parameter	Value
p_n	4 W	ρ^n	5 KB
g	0.2	G	1
ξ_S	0.1	σ^n	20 cycles/bit
$B_{n,u}$	5 MB	$B_{n,m}$	2 MB
κ_B	2	$W(nm)$	1
d_S	10 ms	J^S_{max}, J^U_{max}	1000, 1500
x	1.04	y	200

processes: knowledge prediction process and online scheduling process. In knowledge prediction process, satellites and aircrafts apply Algorithm 1 to predict processing rate $c_n(t)$ and $c_u(t)$, respectively, Then, we can determine the predicted knowledge $\hat{P}(t)$ at time slot t (lines 1–6). In online scheduling process, each satellite uses BnB-based method to obtain near-optimal offloading and scheduling policy $\{\boldsymbol{a}_n(t), \boldsymbol{k}_n(t), n \in \mathbb{N}\}$ in a distributed manner (lines 7–25). According to the optimal online offloading and scheduling decisions from the minimization problem (**P3**), one satellite is able to implement task offloading and scheduling at each time slot t. These two processes implement online together and influence each other in order to achieve autonomous coordination among satellites and aircrafts.

VI. NUMERICAL RESULTS

In this section, we evaluate the performance of the proposed offloading and scheduling methods in MEC deployed SAGIN. We consider a 50×50 km^2 area with 20 aircrafts and 10 LEO satellites distributed by homogeneous Poisson Point Process at space and air domains, respectively. The service limitation of the nth satellite $\chi_n(t)$ and the uth aircraft $\chi_u(t)$ are uniformly distributed in $[80, 100]$, $[120, 140]$, respectively. Other evaluation parameters are listed in Table I.

We compare the performance of the proposed OOS mechanism with three benchmarks: 1) Local Computation (LC): satellites can only locally executes the arrived tasks by themselves. 2) Cooperative Computation with Satellites (CCS): satellites can either executes the tasks by themselves or offload the tasks to the neighboring satellites for processing. 3) Cooperative Computation with Aircrafts (CCA): besides local computation, satellites randomly select one aircraft for task computation.

A. Task Completion Delay

Fig. 2 presents the completion delay comparison of the proposed OOS method, LC method, CCS and CCA method. It can be observed that the proposed OOS method achieves the lowest completion delay network in four methods. The intuitive is that the proposed OOS method is able to offload computation tasks multiple hops to the powerful cloud servers or one-hop to the aircrafts, which results in low computation latency. In LC method, the satellites can only process the computation tasks locally under limited computing capacity which leads to high computation latency. In CCA method, satellites ignore the

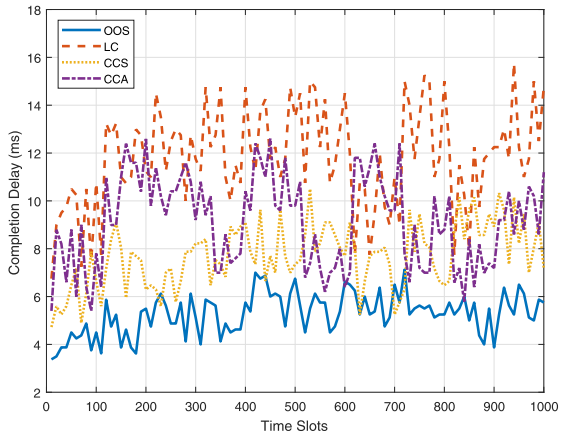

Fig. 2. Completion delay comparison of the space/aerial-aided MEC network.

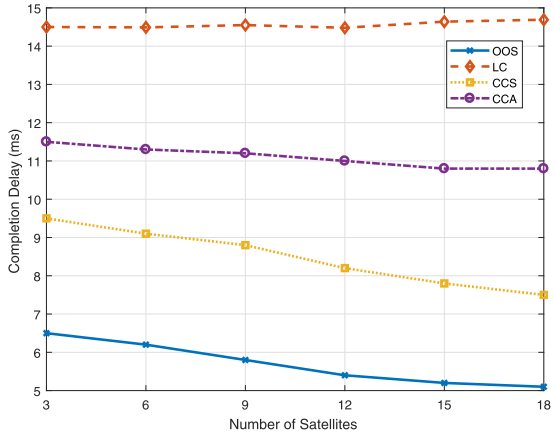

Fig. 3. Completion delay comparison of the space/aerial-aided MEC network.

computation cooperation among satellites and computation capacities among aircrafts when making offloading decisions, thus leading to higher completion delay. In addition, the performance of CCS method is inferior to the OOS method due to failure of exploiting computing capability of cloud server, even with low transmission latency.

Figs. 3 and 4 demonstrate the comparison of completion delay in terms of the number of satellites and cooperative aircrafts, respectively. We can see that in both Figs. 3 and 4, the proposed OOS method still achieves lowest completion delay than that of other methods. That is, the tasks are more likely to be offloaded to the cloud server or powerful aircrafts for processing. As expected, the delay caused by LC method is not effected by the variation of the number of the satellites and aircrafts since the LC method can only process the tasks locally. In Fig. 3, we can observe that the completion delay of CCS method decreases more evidently than that of CCA method as the number of satellites increases. This is because more satellites can provide more opportunities to choose more powerful satellite for task processing. For the same reason, the completion delay of CCA method has obvious decline trend compared to CCS method as the number of aircrafts increases in Fig. 4.

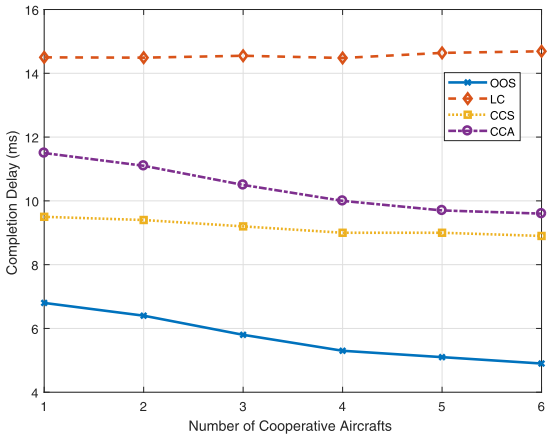

Fig. 4. Completion delay comparison of the space/aerial-aided MEC network.

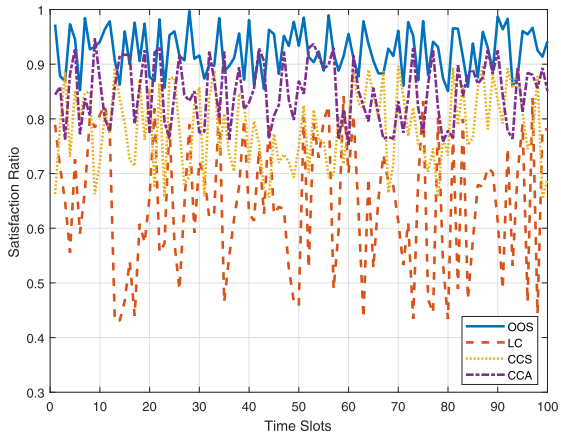

Fig. 6. Completion rate comparison in terms of time slots.

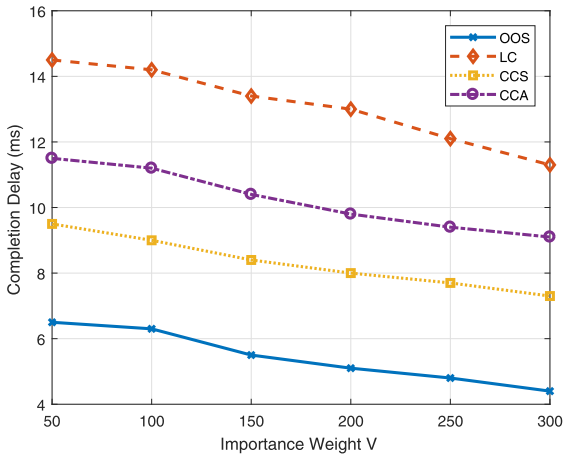

Fig. 5. Completion delay comparison of the space/aerial-aided MEC network.

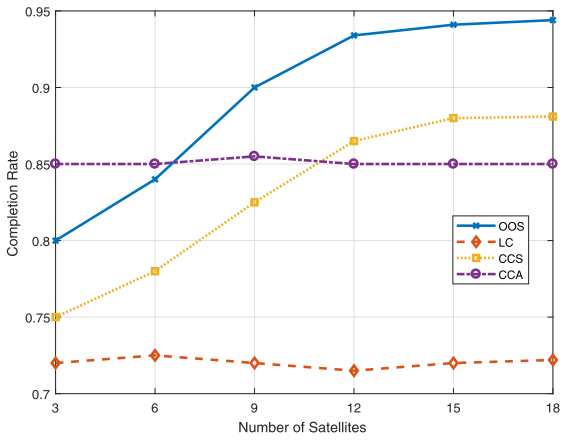

Fig. 7. Completion rate comparison in terms of number of satellites.

In Fig. 5, the completion delay caused by four methods decrease as V-value increases. The reason is that high V value indicates reducing completion delay more in online offloading control. It can be seen that the proposed OOS method causes the lowest delay. That is, the OOS method is superior in achieving a trade-off between the queue stability and task completion delay by employing optimal task offloading and scheduling strategy. Moreover, although the completion delay in four methods are all decreasing with V value, the LC method has the most dramatic decline trend. This is because with no cooperation with other satellites or aircrafts, LC method is more sensitive to V value in balancing queue stability and task completion delay.

B. Completion Rate Comparison

Next, we will compare the completion rate which is defined as the proportion of successful processing tasks that are able to meet the deadlines. In Fig. 6, the completion rate of the proposed OOS, LC, CCS and CCA methods are illustrated in terms of the time slots. We can see that the proposed OOS method is able to achieve the highest completion rate due to superior offloading controls and collaborative computing potentials. LC method has the lowest completion rate. That is, the LC method only employs the local computation which is difficult to process all tasks alone in resource constrained satellites. Compared to proposed OOS and CCA, the completion rate of CCS is lower since it fails to fully exploit computing resources of all neighboring satellites and aircrafts. CCA achieves similar completion rate to OLCD with small gap. Because offloading control in CCA is performed by selecting cooperative aircrafts with adequate resources. It indicates that historic interactions can partly reflect the quality of current offloading service offering.

Fig. 7 shows the completion rate of four offloading methods in terms of the number of satellites. The completion rate of the LC and CCA methods are basically not effected by the increase of the number of the satellites. That's because the task offloading and scheduling in both methods do not need satellites' participation. On the contrary, the completion rate achieved by the proposed OOS and CCS methods are increasing as the increment of number of satellites. That is, the OOS method can obtain higher offloading performance from more multi-hop satellites' paths. And the CCS method has a higher possibility to select the powerful satellite for task processing as the satellites' number grows. Note that the completion rate of the proposed OOS with optimal task offloading scheduling strategies is higher

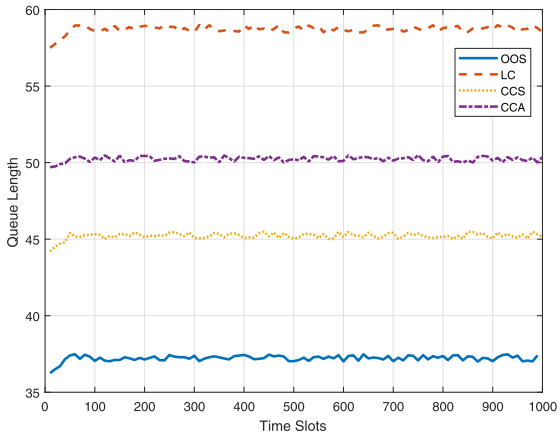

Fig. 8. Average Queue Length comparison in terms of time slots.

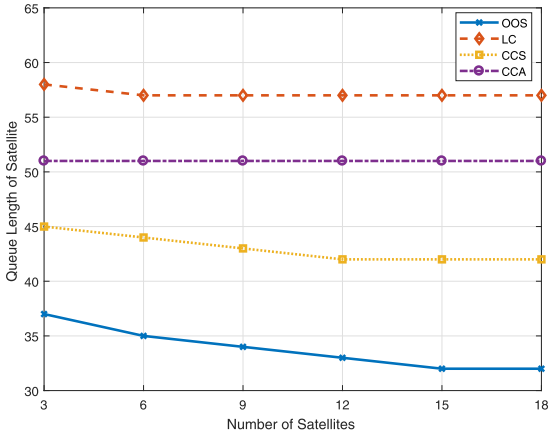

Fig. 9. Average Queue Length comparison in terms of number of satellites.

than that of the CCS method which can only offload tasks to one satellite for processing.

C. Average Queue Length Comparison

Fig. 8 shows the comparison of the average queue length of the satellites in the proposed OOS method, LC, CCS and CCA methods. It can be seen that these methods basically maintain their task queues at steady levels. This is because the Lyapunov optimization in the online offloading control can adaptively balance the queue convergence and system stability. Moreover, considering the stochastic task arrivals and workload levels, the online offloading methods lead to task queues fluctuate around fixed values. We also observe that the LC method has the highest queue length. That's because in the LC method, satellite can only process task locally and it is hard to execute all tasks which lead to larger task queue backlogs. Other three online offloading methods have smaller queue backlogs, because of capability of offloading more tasks to neighboring satellites and aircrafts for processing.

Fig. 9 shows the average queue length in terms of the number of satellites. It can be observed that queue length of the proposed OOS and CCS methods decrease as the number of satellites increases. It is reasonable that larger number of satellites implies higher probability to employ more powerful satellites for multi-hop offloading in OOS method or processing in CCS method, respectively. We also observe that the proposed OOS offloading method has the lower queue length than that of the CCS method, since this method not only choose the most powerful aircraft but employ cloud server through multi-hop satellite path for task processing. Still, we can see that the trends of queue length in both LC and CCA methods are nearly not changed. That' because these two methods can handle the task processing by local computing or aircrafts' cooperation which can avoid extra invoking of satellites.

VII. Conclusion

In this article, we propose a collaborative space/aerial MEC framework for the 6G system in which the LEO satellite is treated not only as a computation server but for the first time as a consumer. The resource-limited satellites obtain offloading and scheduling decisions via the long-term completion delay minimization problem. Without knowing future knowledge of the complex space/aerial environment, a delayed online learning method is resorted to predict task arrival and queue length of satellites and cooperative aircrafts. By using the predicted results as input, the Lyapunov-based optimization and bounded integer programming are jointly used to obtain the solutions of the proposed problem. The numerical results validate the effectiveness of the proposed space/aerial-aided offloading method.

References

[1] F. Tariq, M. R. A. Khandaker, K. -K. Wong, M. A. Imran, M. Bennis, and M. Debbah, "A speculative study on 6G," *IEEE Wireless Commun.*, vol. 8, pp. 118–125, Aug. 2020.

[2] X. You et al., "Towards 6G wireless communication networks: Vision, enabling technologies, and new paradigm shifts," *Sci. China*, vol. 64, pp. 110301:1–110301:74, Jan. 2021.

[3] F. Tang, Y. Kawamoto, N. Kato, and J. Liu, "Future intelligent and secure vehicular network toward 6G: Machine-learning approaches," *Proc. IEEE*, vol. 108, no. 2, pp. 292–307, Feb. 2020.

[4] S. Fu, J. Gao, and L. Zhao, "Collaborative multi-resource allocation in terrestrial-satellite network towards 6G," *IEEE Trans. Veh. Technol.*, vol. 20, no. 11, pp. 7057–7071, Nov. 2021.

[5] J. Liu, Y. Shi, Z. Md Fadlullah, and N. Kato, "Space-air-Ground integrated network: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2714–2741, Fourthquarter 2018.

[6] N. Kato et al., "Optimizing space-air-ground integrated networks by artificial intelligence," *IEEE Wireless Commun.*, vol. 26, no. 4, pp. 140–147, Aug. 2019.

[7] L. Bai, R. Han, J. Liu, J. Choi, and W. Zhang, "Relay-aided random access in space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 27, no. 6, pp. 37–43, Dec. 2020.

[8] H. Wu et al., "Resource management in space-air-ground integrated vehicular networks: SDN control and AI algorithm design," *IEEE Wireless Commun.*, vol. 27, no. 6, pp. 52–60, Dec. 2020.

[9] Y. Sun, M. Peng, S. Zhang, G. Lin, and P. Zhang, "Integrated satellite-terrestrial networks: Architectures, key techniques, and experimental progress," *IEEE Netw.*, vol. 36, no. 6, pp. 191–198, Nov./Dec. 2022.

[10] Y. Liang, J. Tan, H. Jia, J. Zhang, and L. Zhao, "Realizing intelligent spectrum management for integrated satellite and terrestrial networks," *J. Commun. Inf. Netw.*, vol. 6, no. 1, pp. 32–43, 2021.

[11] S. Fu, J. Gao, and L. Zhao, "Integrated resource management for terrestrial-satellite systems," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3256–3266, Mar. 2020.

[12] C. You, K. Huang, H. Chae, and B. -H. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 16, no. 3, pp. 1397–1411, Mar. 2017.

[13] Y. Liu, S. Xie, Q. Yang, and Y. Zhang, "Joint computation offloading and demand response management in mobile edge network with renewable energy sources," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15720–15730, Dec. 2020.
[14] X. Lyu et al., "Optimal schedule of mobile edge computing for Internet of Things using partial information," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2606–2615, Nov. 2017.
[15] Y. Liu, S. Xie, and Y. Zhang, "Cooperative Offloading and Resource Management for UAV-Enabled Mobile Edge Computing in Power IoT System," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 12229–12239, Oct. 2020.
[16] W. Feng, S. Lin, N. Zhang, G. Wang, B. Ai, and L. Cai, "Joint C-V2X based offloading and resource allocation in multi-tier vehicular edge computing system," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 432–445, Feb. 2023.
[17] W. Feng et al., "Energy-efficient collaborative offloading in NOMA-Enabled fog computing for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13794–13807, Aug. 2022.
[18] Y. Chen, B. Ai, Y. Niu, H. Zhang, and Z. Han, "Energy-constrained computation offloading in space-air-ground integrated networks using distributionally robust optimization," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12113–12125, Nov. 2021.
[19] C. Zhou et al., "Deep reinforcement learning for delay-oriented IoT task scheduling in space-air-ground integrated network," *IEEE Trans. On Wireless Commun.*, vol. 20, no. 2, pp. 911–925, Feb. 2021.
[20] N. Cheng et al., "Space/Aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
[21] S. Yu, X. Gong, Q. Shi, X. Wang, and X. Chen, "EC-SAGINs: Edge computing-enhanced space-air-ground integrated networks for internet of vehicles," *IEEE Internet Things*, vol. 9, no. 8, pp. 5742–5754, Apr. 2022.
[22] Y. Liu, L. Jiang, Q. Qi, and S. Xie, "Energy-efficient space-air-ground integrated edge computing for Internet of Remote Things: A. federated DRL approach," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 4845–4856, Mar. 2023.
[23] H. Liao, Z. Zhou, X. Zhao, and Y. Wang, "Learning-based queue-aware task offloading and resource allocation for space-air-ground-integrated power IoT," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5250–5263, Apr. 2021.
[24] Y. Li, X. Wang, X. Gan, H. Jin, L. Fu, and X. Wang, "Learning-aided computation offloading for trusted collaborative mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 12, pp. 2833–2849, Dec. 2020.
[25] M. J. Neely, *Stochastic Network Optimization With Application to Communication and Queueing Systems.*, San Mateo, CA, USA: Morgan Claypool, 2010.
[26] N. Chen, A. Agarwal, A. Wierman, S. Barman, and L. L. H. Andrew, "Online convex optimization using predictions," in *Proc. ACM SIGMETRICS Int. Conf. Meas. Model., Comput. Syst.*, 2015, pp. 191–204.
[27] K. Quanrud and D. Khashabi, "Online learning with adversarial delays," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 1270–1278.
[28] J. Liu, J. Zhou, M. S. Kamel, and X. Luo, "Online learning algorithm based on adaptive control theory," *IEEE Trans. Neural Learn. Syst.*, vol. 29, no. 6, pp. 2278–12239, Jun. 2018.

Yi Liu received the Ph.D. degree from South China University of Technology (SCUT), Guangzhou, China, in 2011. He joined the Singapore University of Technology and Design (SUTD), Singapore, as a Postdoctoral. In 2014, he was with the Institute of Intelligent Information Processing, Guangdong University of Technology, Guangzhou, China, where he is currently a Full Professor. His research interests include wireless communication networks, cooperative communications, and smart grid and intelligent edge computing.

Li Jiang (Member, IEEE) received the Ph.D. degree from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2017. From 2015 to 2016, she was with the University of Oslo, Oslo, Norway, and with the Simula Metropolitan Center for Digital Engineering, Norway, as a Visiting Ph.D. Student, respectively. She is currently a Lecturer with the School of Automation, Guangdong University of Technology (GDUT), Guangzhou, China. Her research interests include resource management for B5G and 6G networks, mobile blockchains, mobile edge computing, and distributed machine learning.

Qi Qi (Senior Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2010. She is currently a Professor with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. She has authored or coauthored more than 100 papers in international journal. Her research interests include network intelligence, edge intelligence, UAV network, cloud computing, distributed deep learning, and deep reinforcement learning. She was the recipient of three National Natural Science Foundations of China.

Kan Xie (Member, IEEE) received the Ph.D. degree in control science and engineering from the Guangdong University of Technology, Guangzhou, China, in 2017. He joined the Institute of Intelligent Information Processing, Guangdong University of Technology, where he is currently an Associate Professor. His research interests include machine learning, nonnegative signal processing, blind signal processing, smart grid, and Internet of Things.

Shengli Xie (Fellow, IEEE) received the M.S. degree in mathematics from Central China Normal University, Wuhan, China, in 1992, and the Ph.D. degree in automatic control from the South China University of Technology, Guangzhou, China, in 1997. From 2006 to 2010, he was the Vice Dean with the School of Electronics and Information Engineering, South China University of Technology, China. He is currently the Director with the Institute of Intelligent Information Processing (LI2P) and with the Guangdong Key Laboratory of Information Technology for the Internet of Things, and a Professor with the School of Automation, Guangdong University of Technology, Guangzhou, China. He has authored or co-authored four monographs and more than 100 scientific papers published in journals and conference proceedings, and was granted more than 30 patents. His research interests include statistical signal processing and wireless communications, with an emphasis on blind signal processing, and Internet of things.